

Batch – T5**Assignment No. – 4****Title – Divide and Conquer Strategy – Part 2****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011****Strassen's Matrix Multiplication**1) Implement Naive method multiply two matrices and Justify complexity is $O(n^3)$ **Algorithm:**

Given two matrices:

- AAA of size $n \times m$
- BBB of size $m \times p$

The resulting matrix C will be of size $n \times p$.For each element in the resulting matrix $C[i][j]$:

1. Initialize $C[i][j]$ to 0.
2. For each k from 0 to $m - 1$:
 Multiply $A[i][k]$ by $B[k][j]$ and add the result to $C[i][j]$.

Pseudocode:

Input:

A, a matrix of size $n \times m$ B, a matrix of size $m \times p$

Output:

C, a matrix of size $n \times p$ Initialize matrix C of size $n \times p$ with all elements as 0for i from 0 to $n-1$: for j from 0 to $p-1$: $C[i][j] = 0$ for k from 0 to $m-1$: $C[i][j] = C[i][j] + (A[i][k] * B[k][j])$

return C

Code:

```
import java.util.Scanner;
```

```
public class Q1 {  
    public static int[][] multiplyMatrices(int[][] A, int[][] B, int n, int m, int p)  
{  
        int[][] C = new int[n][p];  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < p; j++) {  
                C[i][j] = 0;  
                for (int k = 0; k < m; k++) {  
                    C[i][j] += A[i][k] * B[k][j];  
                }  
            }  
        }  
    }  
}
```

```
    }

    return C;
}

public static void displayMatrix(int[][] matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int m = sc.nextInt();
    int p = sc.nextInt();
    int[][] A = new int[n][m];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            A[i][j] = sc.nextInt();
        }
    }
    int[][] B = new int[m][p];
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < p; j++) {
            B[i][j] = sc.nextInt();
        }
    }
    sc.close();
    int[][] C = multiplyMatrices(A, B, n, m, p);
    displayMatrix(C, n, p);
}
}
```

Test Case 1:

2 2 2
1 2
3 4
5 6
7 8

Output:

19 22
43 50

Test Case 2:

2 3 2

```
1 2 3
4 5 6
7 8
9 10
11 12
```

Output:

```
58 64
139 154
```

Test Case 3:

```
3 3 3
1 0 2
-1 3 1
2 2 2
3 1 2
2 1 1
1 0 1
```

Output:

```
5 1 4
7 2 4
12 4 8
```

Test Case 4:

```
4 2 3
1 2
3 4
5 6
7 8
9 8 7
6 5 4
```

Output:

```
21 18 15
51 42 33
81 66 51
111 90 69
```

2) Implement Strassen's matrix multiplication for 3x3 matrix. Do analysis of algorithm with respect to time complexity.

Strassen's algorithm is an efficient matrix multiplication method that improves the time complexity from $O(n^3)$ (in the naive method) to approximately $O(n^{2.81})$. This is achieved by reducing the number of multiplications.

Algorithm:

1) Divide matrix A and matrix B into submatrices of equal size.

A = A11 A12
A21 A22

B = B11 B12
B21 B22

2) Compute 7 products using the Strassen formula:

$$P1 = (A11 + A22) * (B11 + B22)$$

$$P2 = (A21 + A22) * B11$$

$$P3 = A11 * (B12 - B22)$$

$$P4 = A22 * (B21 - B11)$$

$$P5 = (A11 + A12) * B22$$

$$P6 = (A21 - A11) * (B11 + B12)$$

$$P7 = (A12 - A22) * (B21 + B22)$$

3) Combine the results to compute the four submatrices of the result:

$$C11 = P1 + P4 - P5 + P7$$

$$C12 = P3 + P5$$

$$C21 = P2 + P4$$

$$C22 = P1 - P2 + P3 + P6$$

4) Combine the submatrices C11, C12, C21, C22 into the final result matrix.

5) Since Strassen's algorithm is designed for square matrices with sizes that are powers of 2, we will modify our approach to handle 3×3 matrices by padding them with zeros to convert them to 4×4.

Code:

```
import java.util.Scanner;

public class Q2 {
    public static int[][] add(int[][] A, int[][] B) {
        int n = A.length;
        int[][] result = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                result[i][j] = A[i][j] + B[i][j];
            }
        }
        return result;
    }

    public static int[][] subtract(int[][] A, int[][] B) {
        int n = A.length;
        int[][] result = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                result[i][j] = A[i][j] - B[i][j];
            }
        }
        return result;
    }

    public static int[][] strassenMultiply(int[][] A, int[][] B) {
        int n = A.length;

        if (n == 1) {
            int[][] result = new int[1][1];
```

```
        result[0][0] = A[0][0] * B[0][0];
        return result;
    }

    // Divide the matrices into 4 submatrices
    int newSize = n / 2;
    int[][] A11 = new int[newSize][newSize];
    int[][] A12 = new int[newSize][newSize];
    int[][] A21 = new int[newSize][newSize];
    int[][] A22 = new int[newSize][newSize];

    int[][] B11 = new int[newSize][newSize];
    int[][] B12 = new int[newSize][newSize];
    int[][] B21 = new int[newSize][newSize];
    int[][] B22 = new int[newSize][newSize];

    // Partition the matrix
    for (int i = 0; i < newSize; i++) {
        for (int j = 0; j < newSize; j++) {
            A11[i][j] = A[i][j];
            A12[i][j] = A[i][j + newSize];
            A21[i][j] = A[i + newSize][j];
            A22[i][j] = A[i + newSize][j + newSize];

            B11[i][j] = B[i][j];
            B12[i][j] = B[i][j + newSize];
            B21[i][j] = B[i + newSize][j];
            B22[i][j] = B[i + newSize][j + newSize];
        }
    }

    // Apply Strassen's formula
    int[][] P1 = strassenMultiply(add(A11, A22), add(B11, B22));
    int[][] P2 = strassenMultiply(add(A21, A22), B11);
    int[][] P3 = strassenMultiply(A11, subtract(B12, B22));
    int[][] P4 = strassenMultiply(A22, subtract(B21, B11));
    int[][] P5 = strassenMultiply(add(A11, A12), B22);
    int[][] P6 = strassenMultiply(subtract(A21, A11), add(B11, B12));
    int[][] P7 = strassenMultiply(subtract(A12, A22), add(B21, B22));

    // Combine the results
    int[][] C11 = add(subtract(add(P1, P4), P5), P7);
    int[][] C12 = add(P3, P5);
    int[][] C21 = add(P2, P4);
    int[][] C22 = add(subtract(add(P1, P3), P2), P6);

    // Combine C11, C12, C21, C22 into final result matrix
    int[][] result = new int[n][n];
    for (int i = 0; i < newSize; i++) {
        for (int j = 0; j < newSize; j++) {
```

```
        result[i][j] = C11[i][j];
        result[i][j + newSize] = C12[i][j];
        result[i + newSize][j] = C21[i][j];
        result[i + newSize][j + newSize] = C22[i][j];
    }
}

return result;
}
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = 3;
    int[][] A = new int[4][4];
    int[][] B = new int[4][4];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            A[i][j] = sc.nextInt();
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            B[i][j] = sc.nextInt();
        }
    }

    int[][] result = strassenMultiply(A, B);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.print(result[i][j] + " ");
        }
        System.out.println();
    }

    sc.close();
}
}
```

Test Case 1:

1 2 3
4 5 6
7 8 9
9 8 7
6 5 4
3 2 1

Output:

30 24 18
84 69 54
138 114 90

Test Case 2:

2 4 6
8 10 12
14 16 18
1 2 3
4 5 6
7 8 9

Output:

60 72 84
144 177 210
228 282 336

Test Case 3:

1 0 0
0 1 0
0 0 1
9 8 7
6 5 4
3 2 1

Output:

9 8 7
6 5 4
3 2 1

Test Case 4:

1 1 1
1 1 1
1 1 1
1 1 1
1 1 1
1 1 1

Output:

3 3 3
3 3 3
3 3 3